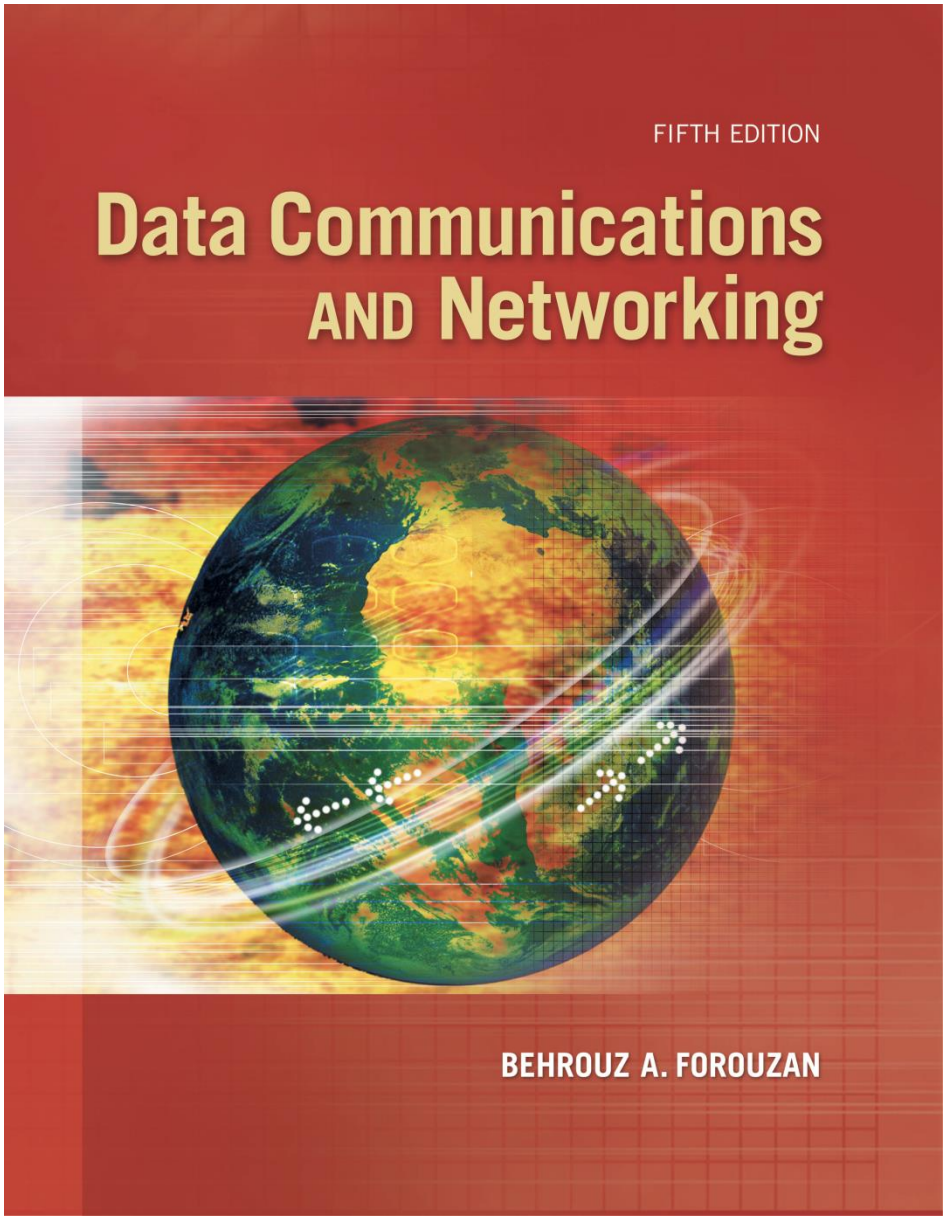
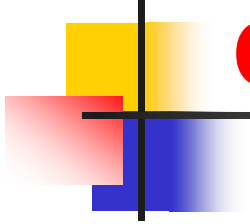
Chapter 25

Introduction To Application Layer



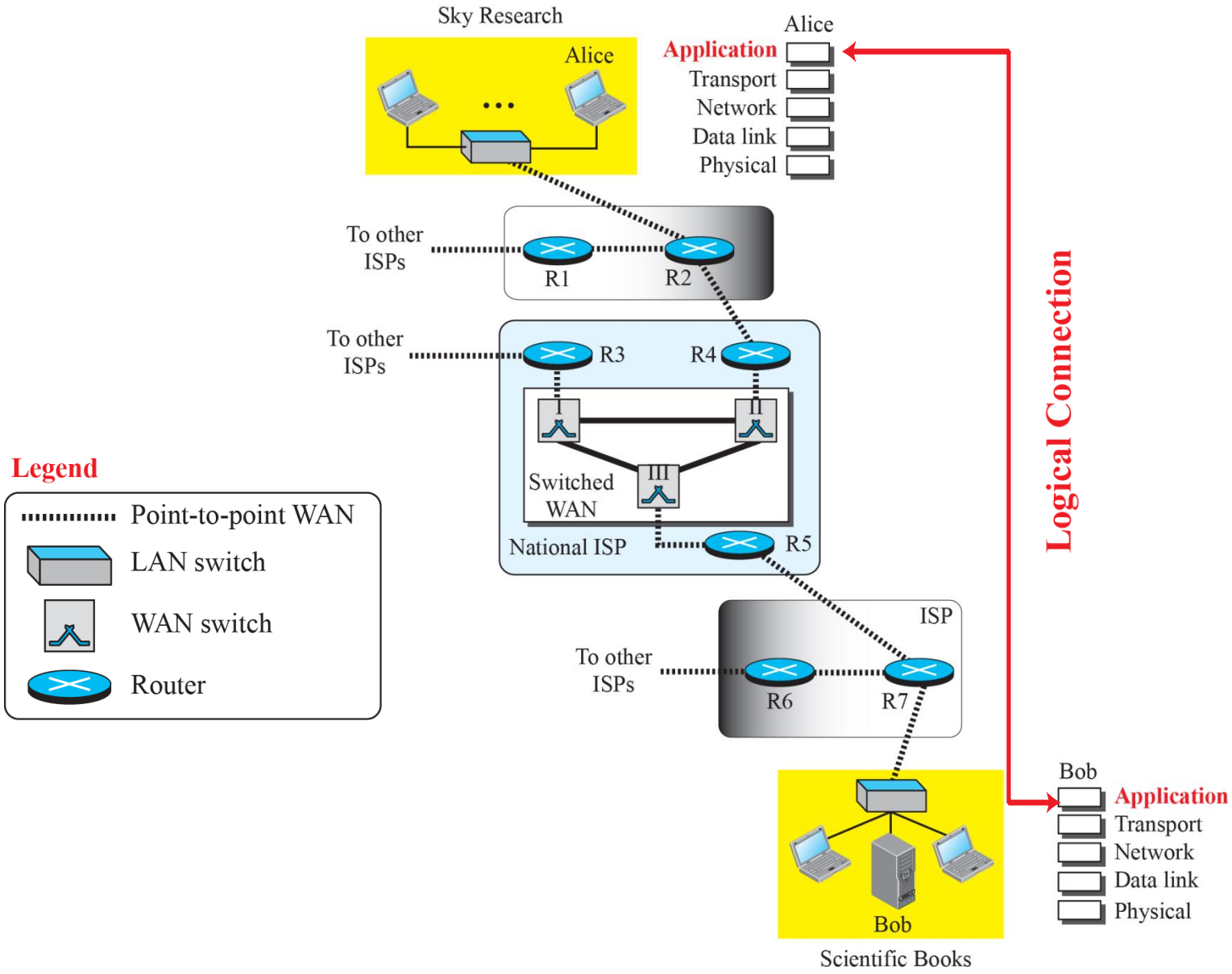
Chapter 25: Outline



25-1 INTRODUCTION

The application layer provides services to the user. Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages. Figure 25.1 shows the idea behind this logical connection.

Figure 25.1: Logical connection at the application layer



25.25.1 Providing Services

All communication networks that started before the Internet were designed to provide services to network users. Most of these networks, however, were originally designed to provide one specific service. For example, the telephone network was originally designed to provide voice service: to allow people all over the world to talk to each other. This network, however, was later used for some other services, such as facsimile (fax), enabled by users adding some extra hardware at both ends.

25.25.2 Application-Layer Paradigms

It should be clear that to use the Internet we need two application programs to interact with each other: one running on a computer somewhere in the world, the other running on another computer somewhere else in the world. The two programs need to send messages to each other through the Internet infrastructure. However, we have not discussed what the relationship should be between these programs. Two paradigms have been developed : the client-server paradigm and the peer-to-peer paradigm. We briefly introduce these two paradigms here.

Figure 25.2: Example of a client-server paradigm

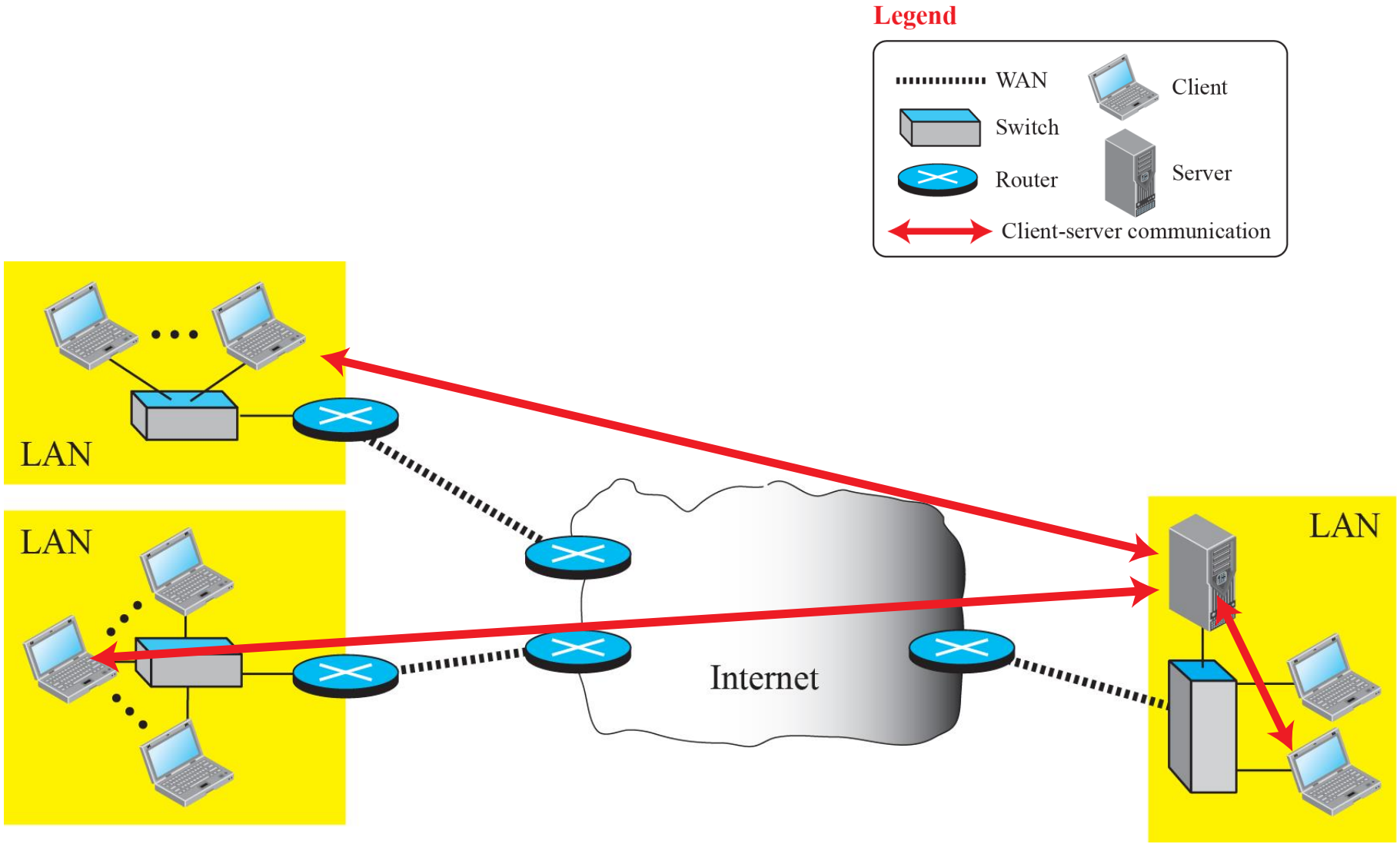
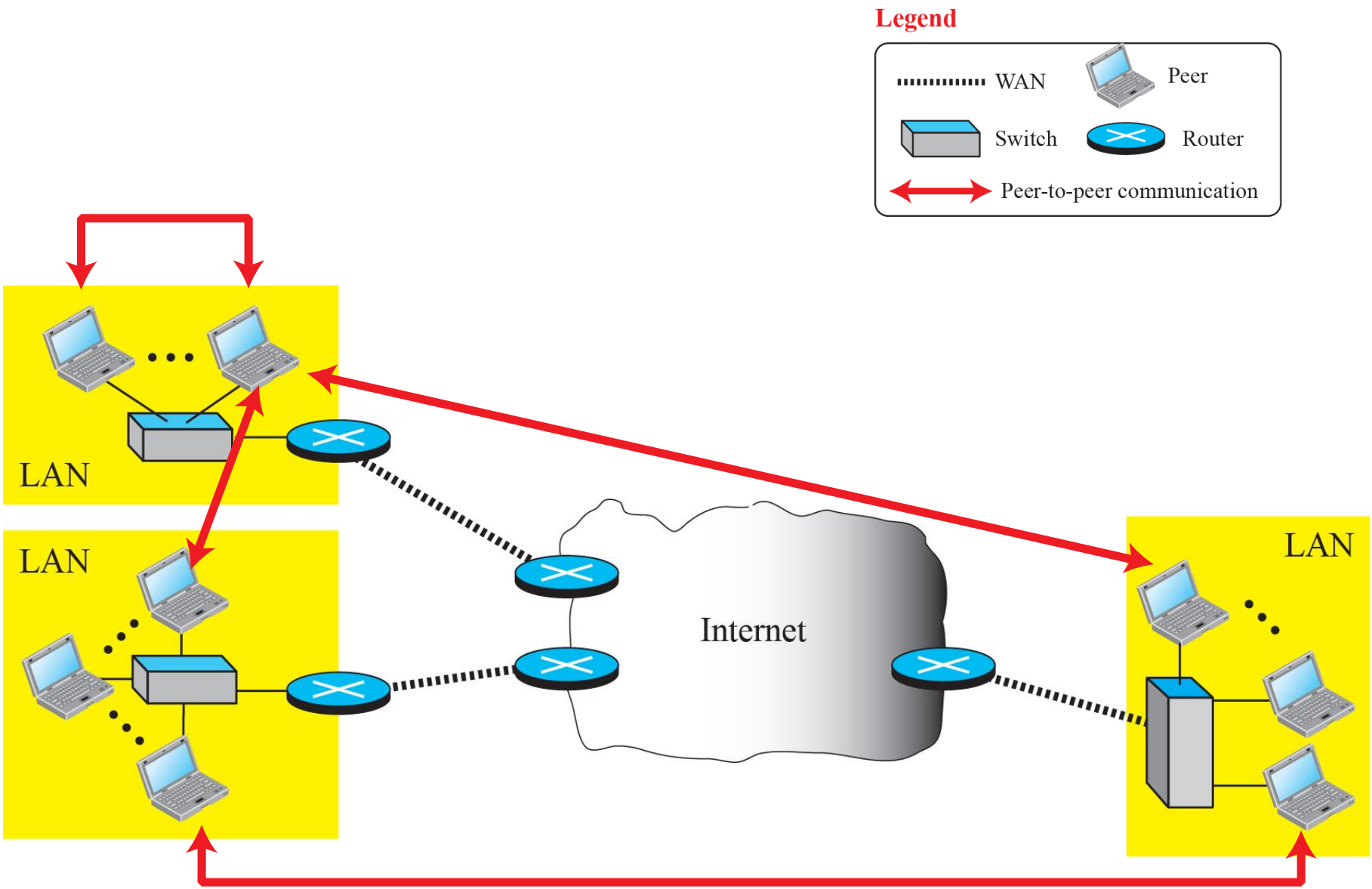


Figure 25.3: Example of a peer-to-peer paradigm



25-2 CLIENT-SERVER PROGRAMMING

In this paradigm, communication at the application layer is between two running application programs called processes: a client and a server. A client is a running program that initializes the communication by sending a request; a server is another application program that waits for a request from a client.

25.2.1 API

How can a client process communicate with a server process? A computer program is normally written in a computer language with a predefined set of instructions that tells the computer what to do. If we need a process to be able to communicate with another process, we need a new set of instructions to tell the lowest four layers of the TCP/IP suite to open the connection, send and receive data from the other end, and close the connection. A set of instructions of this kind is normally referred to as an application programming interface (API).

Figure 25.4: Position of the socket interface

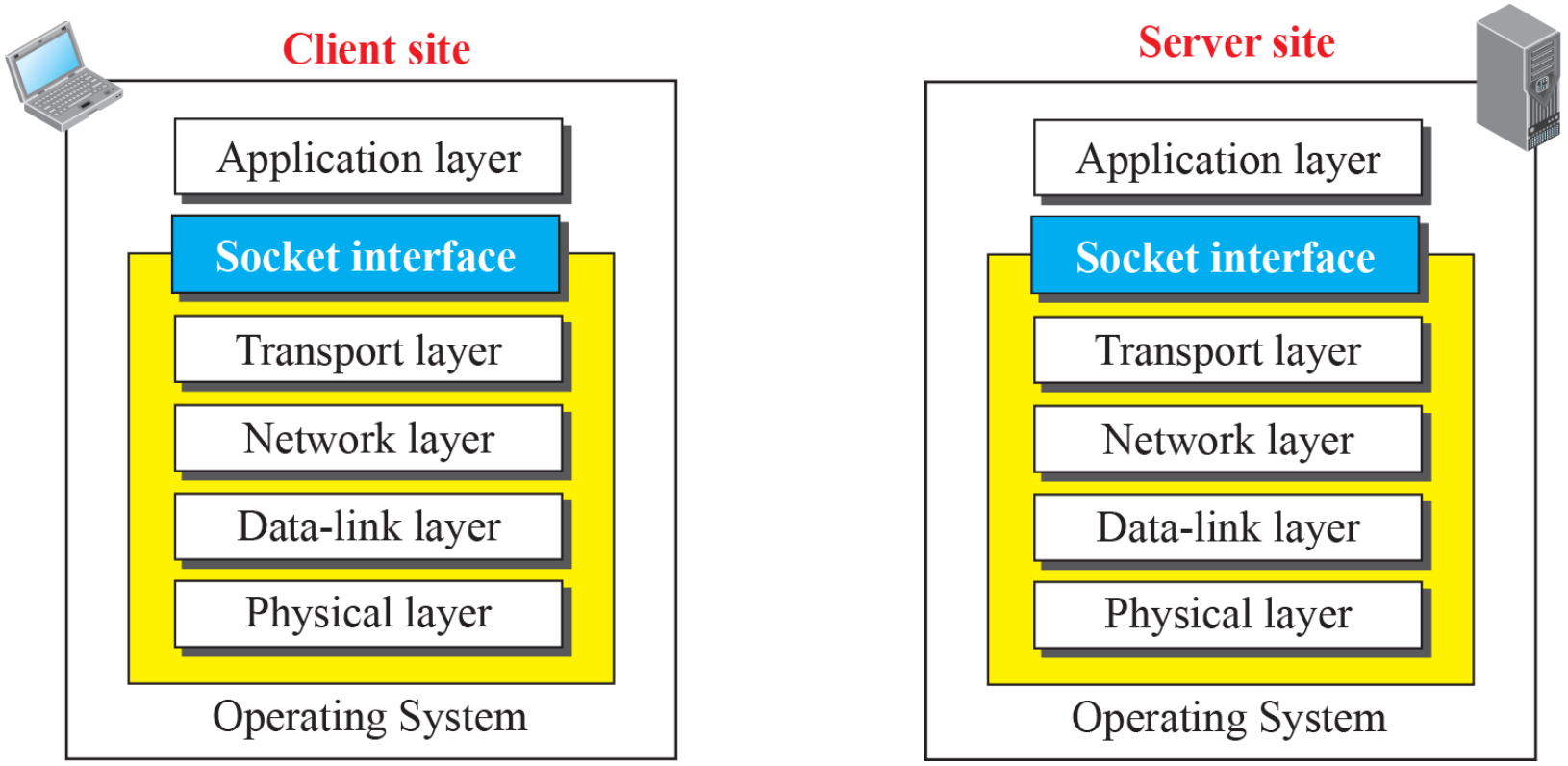


Figure 25.5: A Sockets used like other sources and sinks

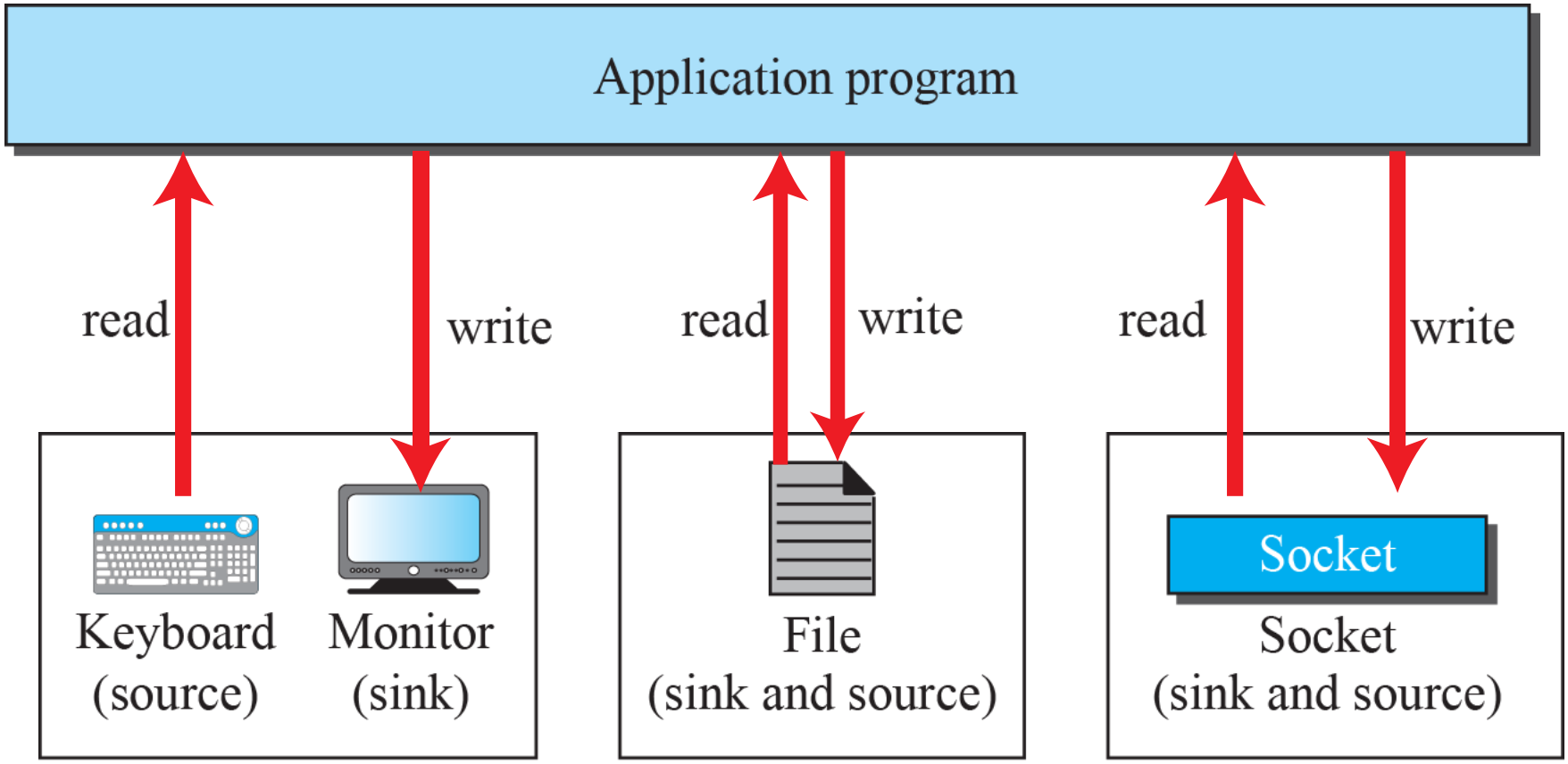


Figure 25.6: Use of sockets in process-to-process communication

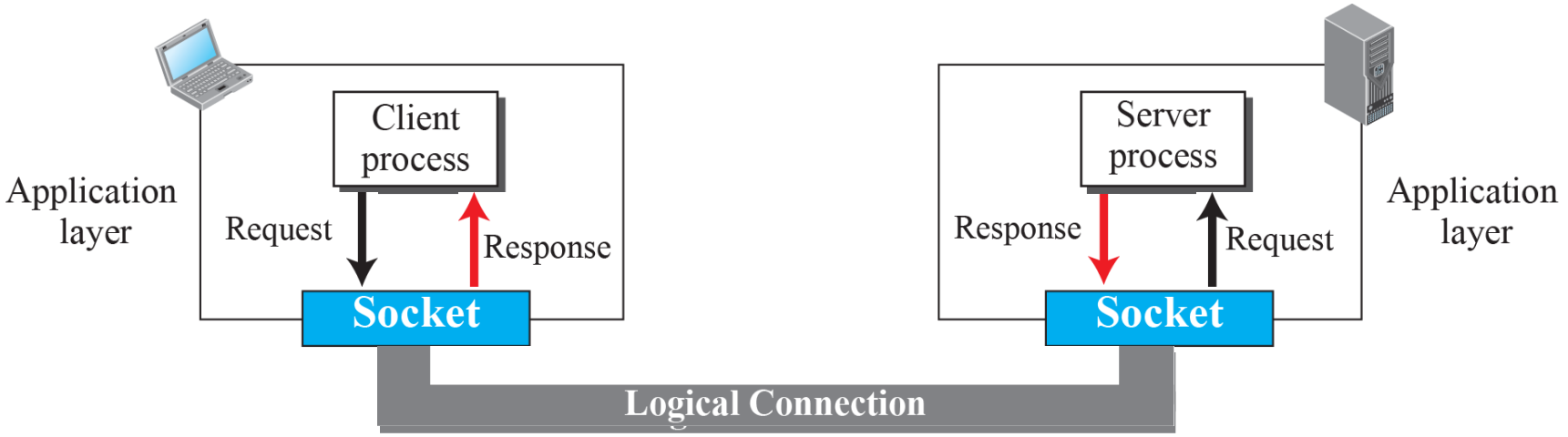


Figure 25.7: A socket address

25.2.2 Using Transport Layer

A pair of processes provide services to the users of the Internet, human or programs. A pair of processes, however, need to use the services provided by the transport layer for communication because there is no physical communication at the application layer. As we discussed in Chapters 23 and 24, there are three common transport-layer protocols in the TCP/IP suite: UDP, TCP, and SCTP. Most standard applications have been designed to use the services of one of these protocols.

25.2.3 Iterative Using UDP

An iterative server can process one client request at a time; it receives a request, processes it, and sends the response to the requestor before handling another request. When the server is handling the request from a client, the requests from other clients, and even other requests from the same client, need to be queued at the server site and wait for the server to be freed. The received and queued requests are handled in the first-in, first-out fashion. In this section, we discuss iterative communication using UDP.

Figure 25.8: Sockets for UDP communication

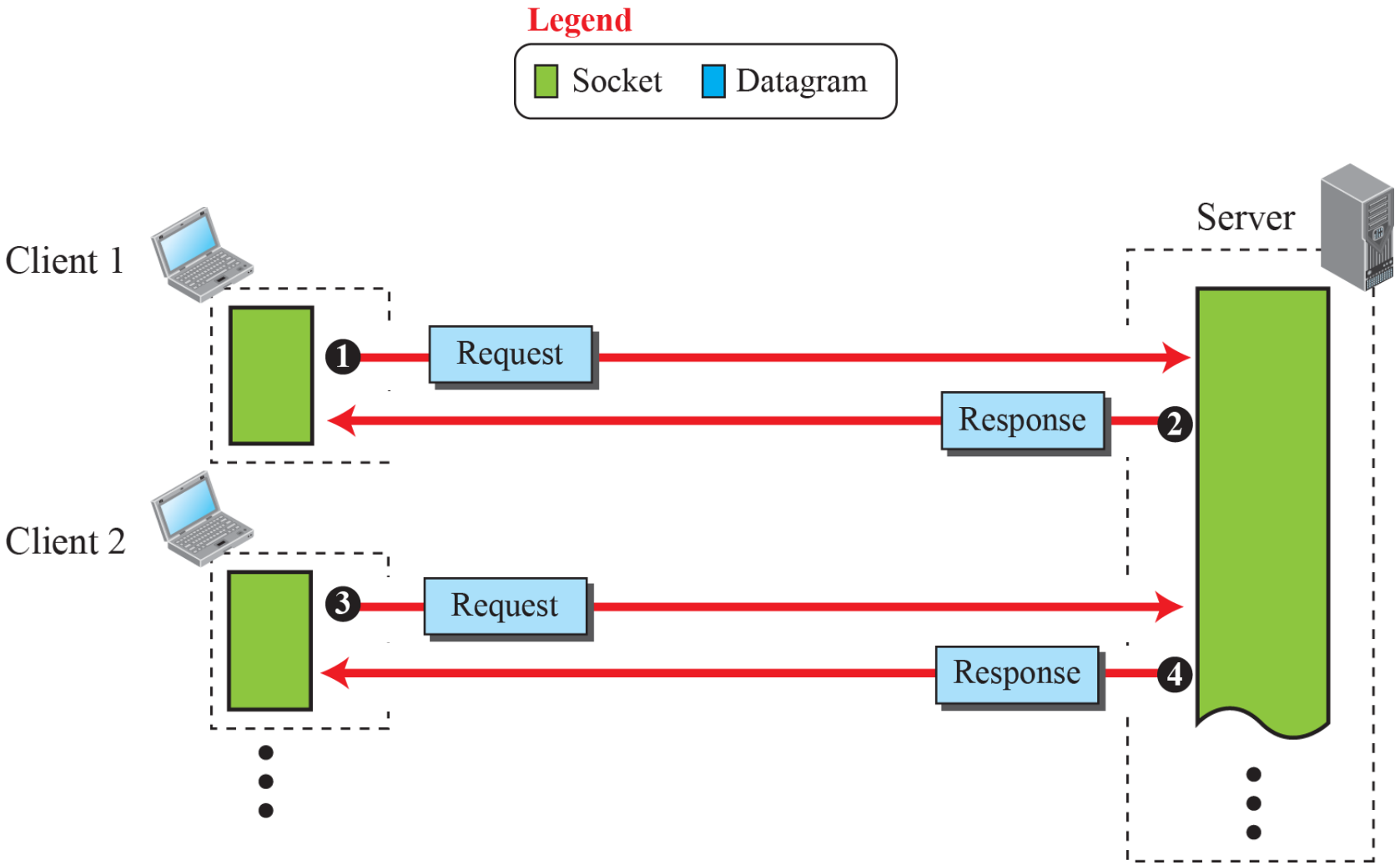
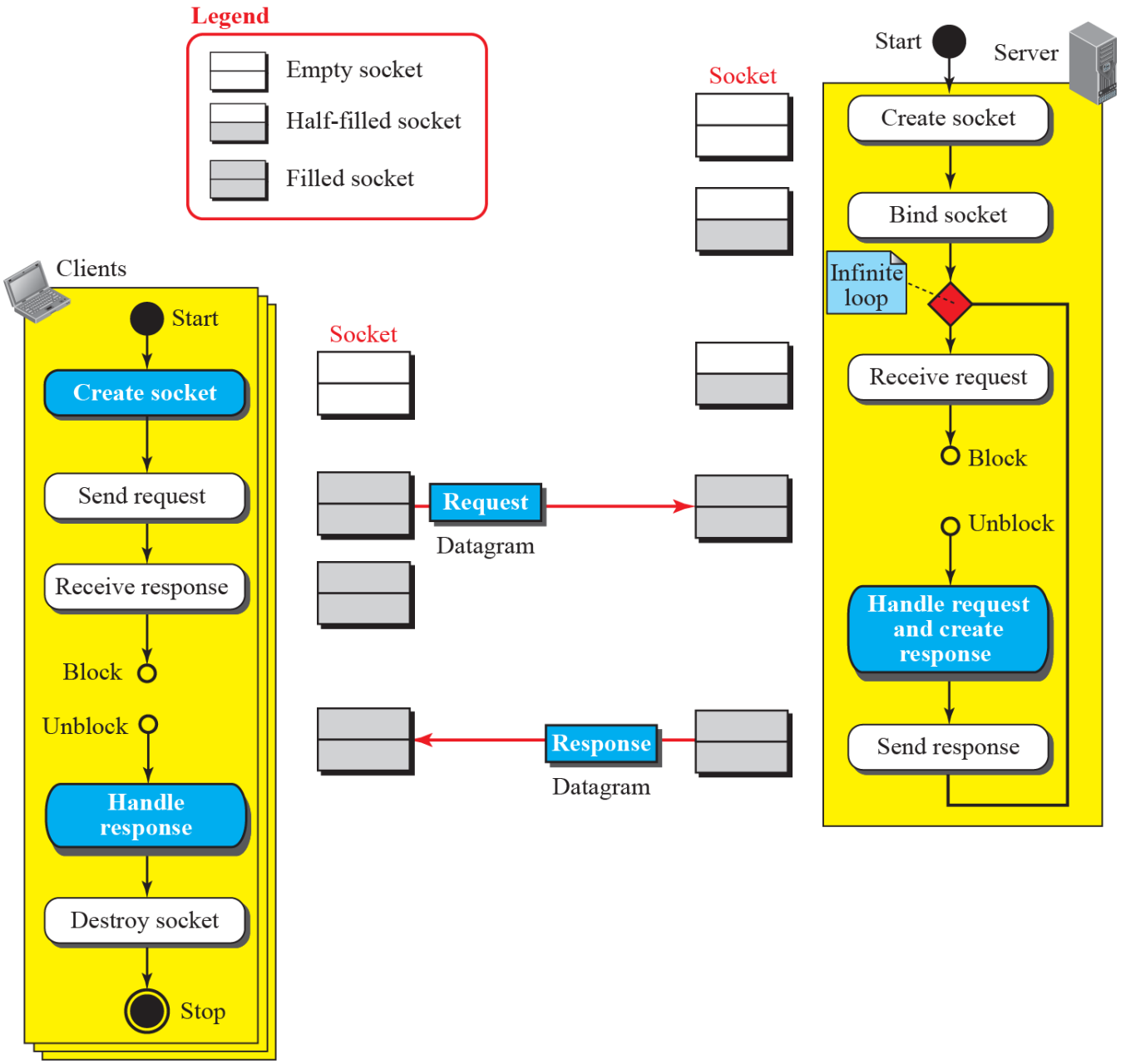


Figure 25.9: Flow diagram for iterative UDP communication



25.2.4 Iterative Using TCP

Although iterative communication using TCP is not very common, because it is simpler we discuss this type of communication in this section.

Figure 25.10: ***Sockets used in TCP communication***

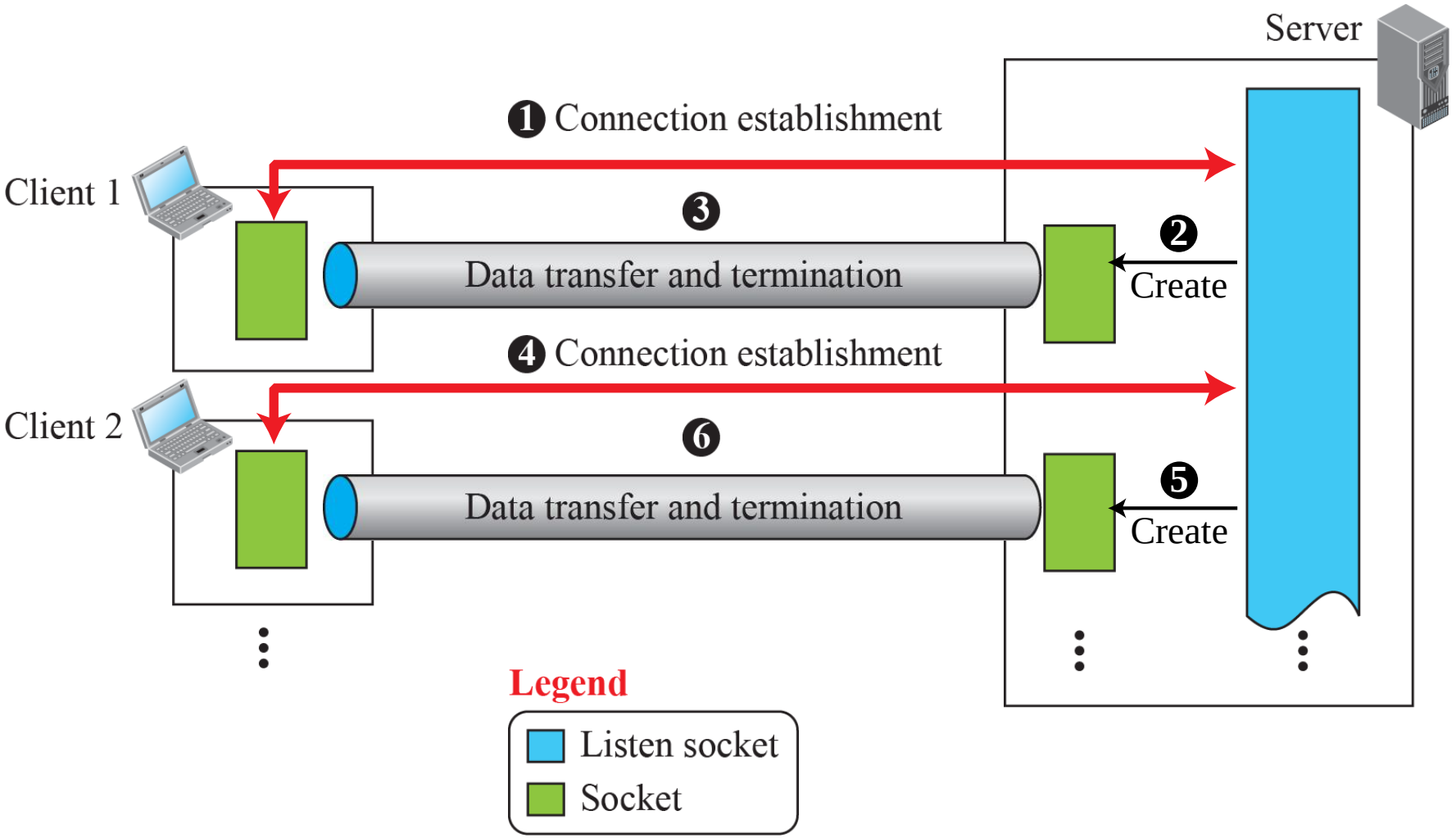
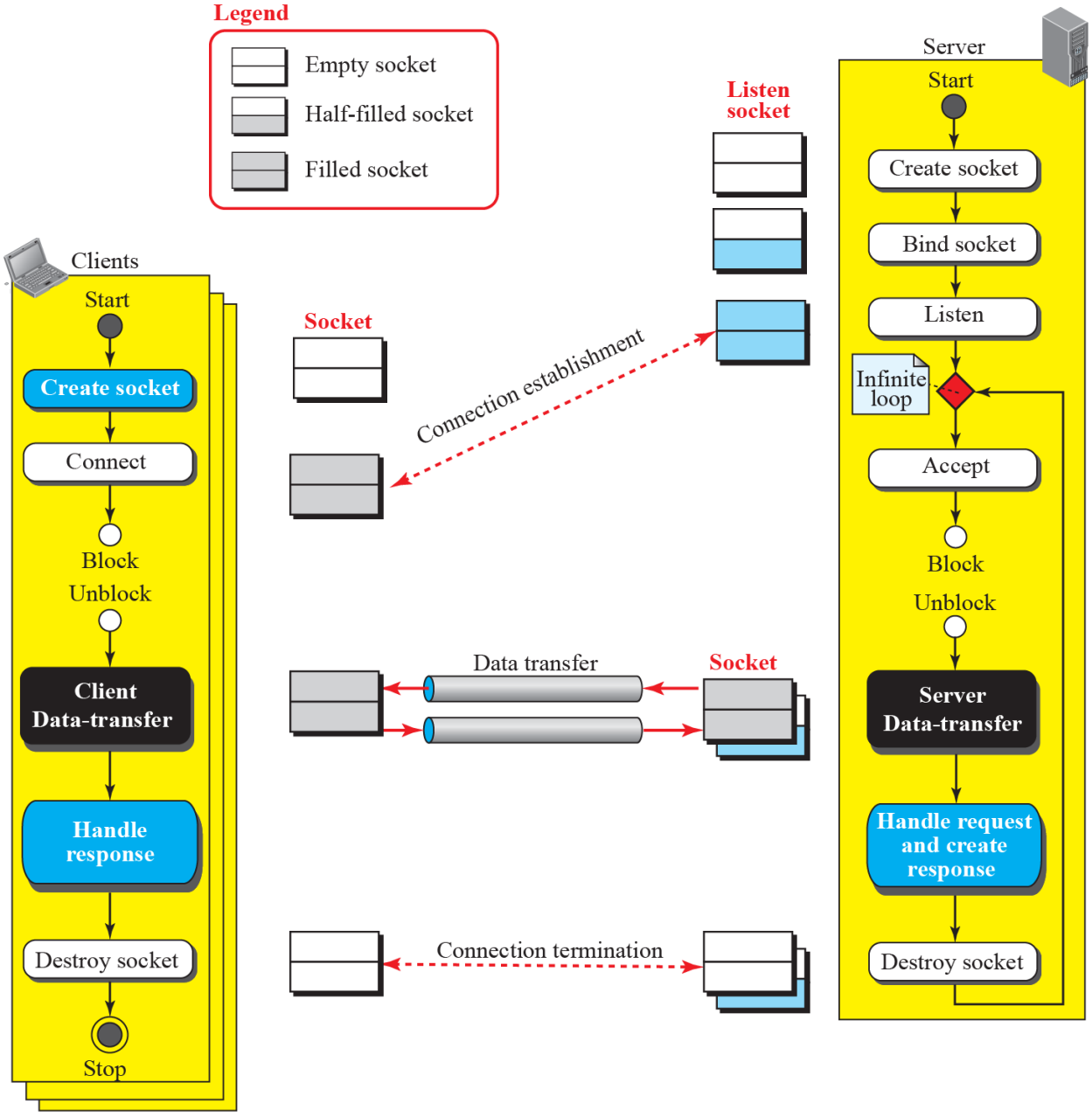


Figure 25.11: Flow diagram for iterative TCP communication



25.2.5 Concurrent Communication

A concurrent server can process several client requests at the same time. This can be done using the available provisions in the underlying programming language. In C, a server can create several child processes, in which a child can handle a client. In Java, threading allows several clients to be handled by each thread. We do not discuss concurrent server communication in this chapter, but we briefly discuss it in the book website in the Extra Material section.

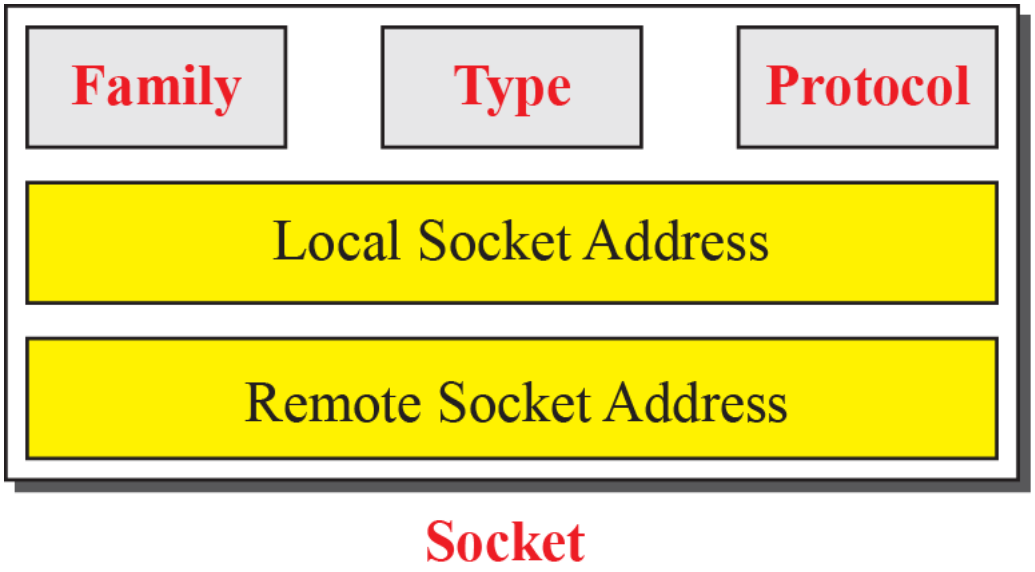
25-3 ITERATIVE PROGRAMMING IN C

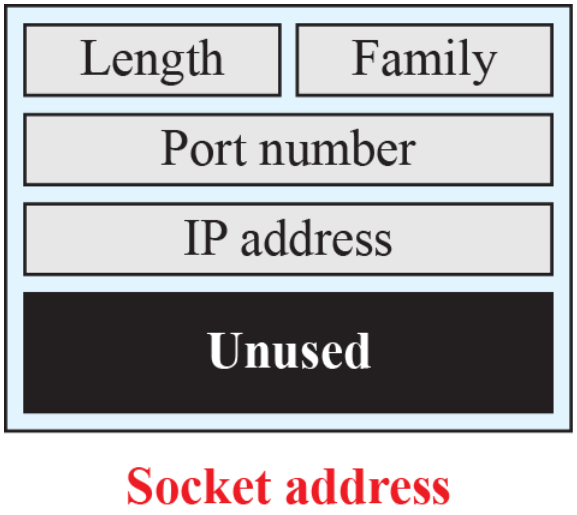
In this section, we show how to write some simple iterative client-server programs using C. The section can be skipped if the reader is not familiar with the C language. The C language better reveals some subtleties in this type of programming.

25.3.1 General Issues

The important issue in socket interface is to understand the role of a socket in communication. The socket has no buffer to store data to be sent or received. It is capable of neither sending nor receiving data. The socket just acts as a reference or a label. The buffers and necessary variables are created inside the operating system.

Figure 25.12: ***Socket data structure***





Header Files

```
// "headerFiles.h"
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>
#include <arpa/innet.h>
#include <sys/wait.h>
```

25.3.2 Iter. Programs Using UDP

As we discussed earlier, UDP provides a connectionless server, in which a client sends a request and the server sends back a response.

Table 25.1: Echo server program using UDP

```
1  // UDP echo server program
2  #include "headerFiles.h"
3  int main (void)
4  {
5      // Declare and define variables
6      int s;                              // Socket descriptor (reference)
7      int len;                            // Length of string to be echoed
8      char   buffer [256];                // Data buffer
9      struct sockaddr_in servAddr;        // Server (local) socket address
10     struct sockaddr_in clntAddr;        // Client (remote) socket address
11     int  clntAddrLen;                   // Length of client socket address
12     // Build local (server) socket address
13     memset (&servAddr, 0, sizeof (servAddr));          // Allocate memory
14     servAddr.sin_family =   AF_INET;                   // Family field
15     servAddr.sin_port = htons (SERVER_PORT)            // Default port number
16     servAddr.sin_addr.s_addr= htonl(INADDR_ANY);       // Default IP address
17     // Create socket
```

Table 25.1: Echo server program using UDP (continued)

```
18      if ((s = socket (PF_INET, SOCK_DGRAM, 0) < 0);
19      {
20              perror ("Error: socket failed!");
21              exit (1);
22      }
23      // Bind socket to local address and port
24      if ((bind (s, (struct sockaddr*) &servAddr, sizeof (servAddr)) < 0);
25      {
26              perror ("Error: bind failed!");
27              exit (1);
28      }
29      for ( ; ; )        // Run forever
30      {
31              // Receive String
32              len = recvfrom (s, buffer, sizeof (buffer), 0,
33                      (struct sockaddr*)&clntAddr, &clntAddrLen);
34              // Send String
35              sendto (s, buffer, len, 0, (struct sockaddr*)&clntAddr, sizeof(clntAddr));
36      } // End of for loop
37  } // End of echo server program
```

Table 25.2: Echo client program using UDP

```
1  // UDP echo client program
2  #include "headerFiles.h"
3  int main (int argc, char* argv[ ])          // Three arguments to be checked later
4  {
5       // Declare and define variables
6       int   s;                               // Socket descriptor
7       int   len;                             // Length of string to be echoed
8       char*  servName;                       // Server name
9       int  servPort;                         // Server port
10      char*  string;                         // String to be echoed
11      char buffer[256 + 1];                  // Data buffer
12      struct sockaddr_in servAddr;           // Server socket address
13      // Check and set program arguments
14      if (argc != 3)
15      {
16           printf ("Error: three arguments are needed!");
17           exit(1);
18      }
19      servName = argv[1];
20      servPort = atoi (argv[2]);
21      string =  argv[3];
22      // Build server socket address
23      memset (&servAddr, 0, sizeof (servAddr));
24      servAddr.sin_family = AF_INET;
25      inet_pton (AF_INET, servName, &servAddr.sin_addr);
```

Table 25.2: Echo client program using UDP (continued)

```
26        servAddr.sin_port = htons (servPort);
27        // Create socket
28        if ((s = socket (PF_INET, SOCK_DGRAM, 0) < 0);
29        {
30              perror ("Error: Socket failed!");
31              exit (1);
32        }
33        // Send echo string
34       len =  sendto (s, string, strlen (string), 0, (struct sockaddr)&servAddr, sizeof
                (servAddr));
35        // Receive echo string
36        recvfrom (s, buffer, len, 0, NULL, NULL);
37        // Print and verify echoed string
38        buffer [len] = '\0';
39        printf ("Echo string received: ";
40        fputs (buffer, stdout);
41        // Close the socket
42        close (s);s
43        // Stop the program
44        exit (0);
45  } // End of echo client program
```

25.3.3 Iter. Programming Using TCP

As we described before, TCP is a connection-oriented protocol. Before sending or receiving data, a connection needs to be established between the client and the server.

Figure 25.13: ***Flow diagram for data-transfer boxes***

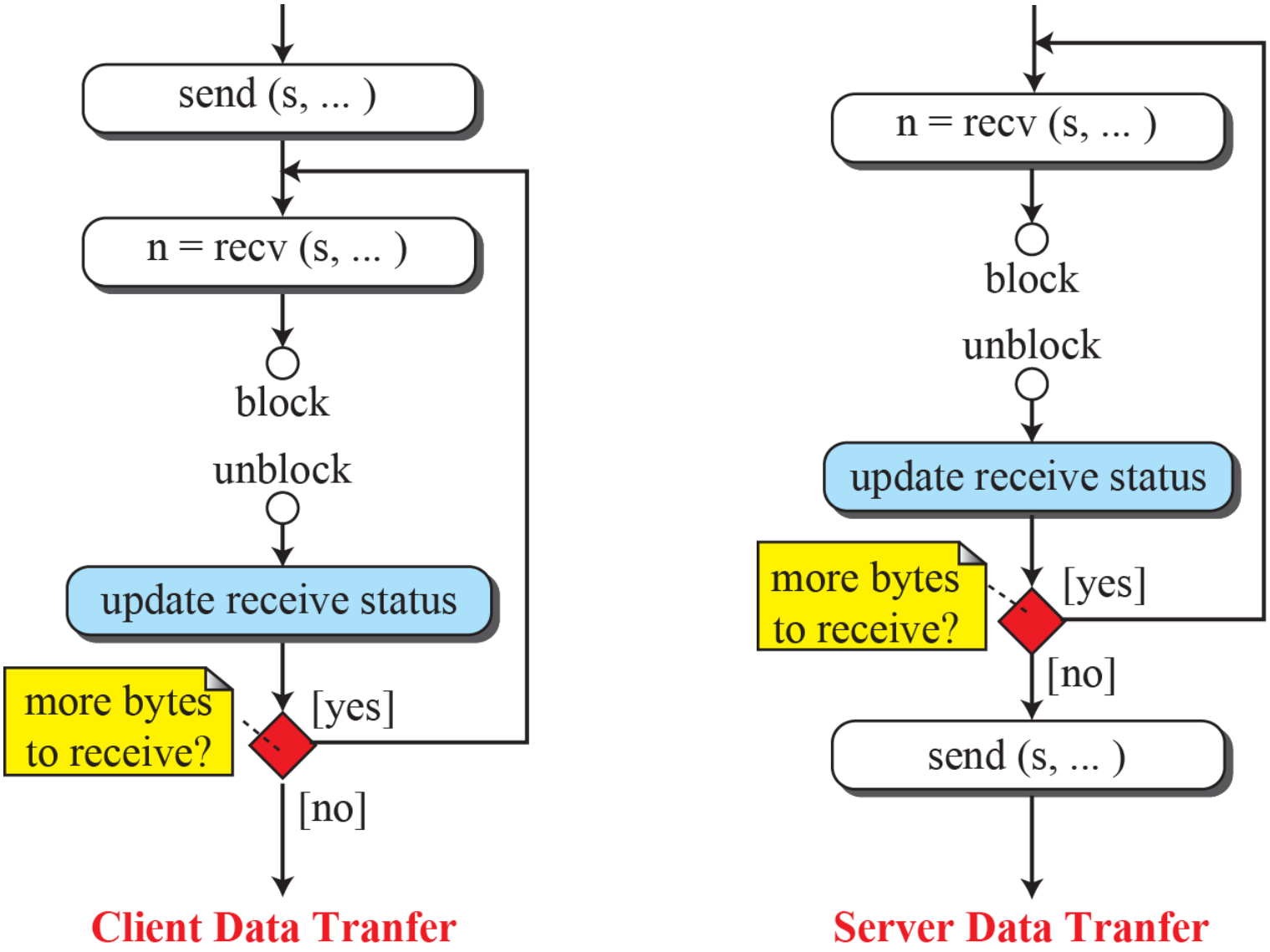


Figure 25.14: ***Buffer used for receiving***

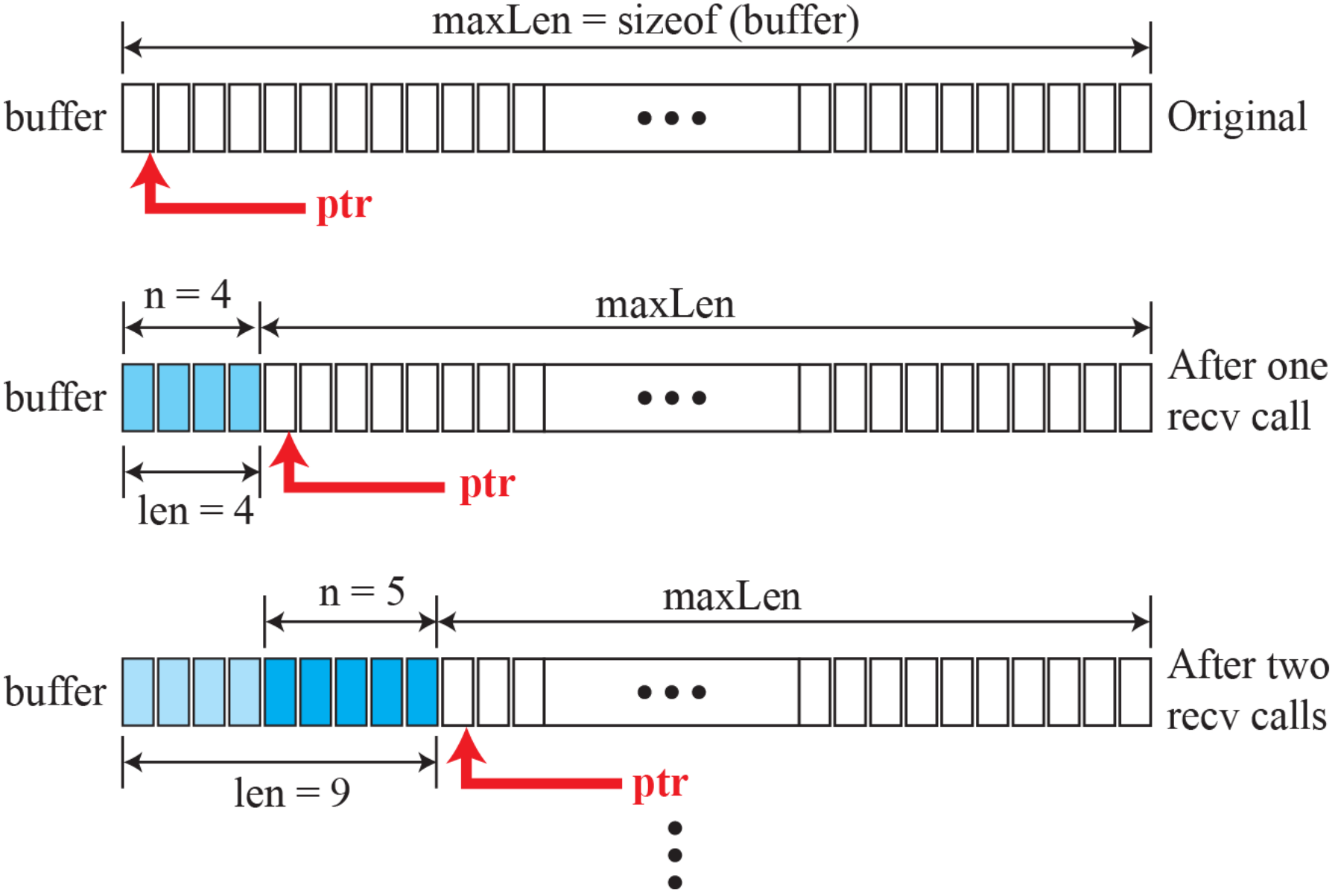


Table 25.3: Echo server program using TCP (part I)

```
1  // Echo server program
2  #include "headerFiles.h"
3  int main (void)
4  {
5      // Declare and define
6      int ls;                          // Listen socket descriptor
7      int s;                           // Socket descriptor (reference)
8      char buffer [256];               // Data buffer
9      char* ptr = buffer;              // Data buffer
10     int len = 0;                     // Number of bytes to send or receive
11     int maxLen = sizeof (buffer);    // Maximum number of bytes
12     int n = 0;                       // Number of bytes for each recv call
13     int waitSize = 16;               // Size of waiting clients
14     struct sockaddr_in serverAddr;   // Server address
15     struct sockaddr_in clientAddr;   // Client address
16     int clntAddrLen;                 // Length of client address
17     // Create local (server) socket address
18     memset (&servAddr, 0, sizeof (servAddr));
19     servAddr.sin_family = AF_INET;
20     servAddr.sin_addr.s_addr = htonl (INADDR_ANY);   // Default IP address
```

Table 25.3: Echo server program using TCP (part II)

```
21      servAddr.sin_port = htons (SERV_PORT);          // Default port
22      // Create listen socket
23      if (ls = socket (PF_INET, SOCK_STREAM, 0) < 0);
24      {
25          perror ("Error: Listen socket failed!");
26          exit (1);
27      }
28      // Bind listen socket to the local socket address
29      if (bind (ls, &servAddr, sizeof (servAddr)) < 0);
30      {
31          perror ("Error: binding failed!");
32          exit (1);
33      }
34      // Listen to connection requests
35      if (listen (ls, waitSize) < 0);
36      {
37          perror ("Error: listening failed!");
38          exit (1);
39      }
```

Table 25.3: Echo server program using TCP (part III)

```
40      // Handle the connection
41      for ( ; ; )                          // Run forever
42      {
43          // Accept connections from client
44          if (s = accept (ls, &clntAddr, &clntAddrLen) < 0);
45          {
46              perror ("Error: accepting failed!);
47              exit (1);
48          }
49          // Data transfer section
50          while ((n = recv (s, ptr, maxLen, 0)) > 0)
51          {
52              ptr + = n;              // Move pointer along the buffer
53              maxLen - = n;           // Adjust maximum number of bytes to receive
54              len + = n;              // Update number of bytes received
55          }
56          send (s, buffer, len, 0);   // Send back (echo) all bytes received
57          // Close the socket
58          close (s);
59      } // End of for loop
60  } // End of echo server program
```

Table 25.4: Echo client program using TCP (Part I)

```
1  // TCP echo client program
2  #include "headerFiles.h"
3  int main (int argc, char* argv[ ])          // Three arguments to be checked later
4  {
5       // Declare and define
6       int  s;                                // Socket descriptor
7       int  n;                                // Number of bytes in each recv call
8       char* servName;                        // Server name
9       int servPort;                          // Server port number
10      char* string;                          // String to be echoed
11      int  len;                              // Length of string to be echoed
12      char  buffer [256 + 1];                // Buffer
13      char* ptr = buffer;                    // Pointer to move along the buffer
14      struct sockaddr_in serverAddr;         // Server socket address
15      // Check and set arguments
16      if (argc != 3)
17      {
18           printf ("Error: three arguments are needed!");
19           exit (1);
20      }
```

Table 25.4: Echo client program using TCP (Part II)

```
21      servName = arg [1];
22      servPort = atoi (arg [2]);
23      string = arg [3];
24      // Create remote (server) socket address
25      memset (&servAddr, 0, sizeof(servAddr);
26      serverAddr.sin_family = AF_INET;
27      inet_pton (AF_INET, servName, &serverAddr.sin_addr); // Server IP address
28      serverAddr.sin_port = htons (servPort);              // Server port number
29      // Create socket
30      if ((s = socket (PF_INET, SOCK_STREAM, 0) < 0);
31      {
32          perror ("Error: socket creation failed!");
33          exit (1);
34      }
35      // Connect  to the server
36      if (connect (sd, (struct sockaddr*)&servAddr, sizeof(servAddr)) < 0);
37      {
38          perror ("Error: connection failed!");
39          exit (1);
40      }
```

Table 25.4: Echo server program using TCP (Part III)

```
41        // Data transfer section
42        send (s, string, strlen(string), 0);
43        while ((n = recv (s, ptr, maxLen, 0)) > 0)
44        {
45              ptr + = n;                    // Move pointer along the buffer
46              maxLen - = n;                 // Adjust the maximum number of bytes
47              len += n;                     // Update the length of string received
48        } // End of while loop
49        // Print and verify the echoed string
50        buffer [len] = '\0';
51        printf ("Echoed string received: ");
52        fputs (buffer, stdout);
53        // Close socket
54        close (s);
55        // Stop program
56        exit (0);
57  } // End of echo client program
```